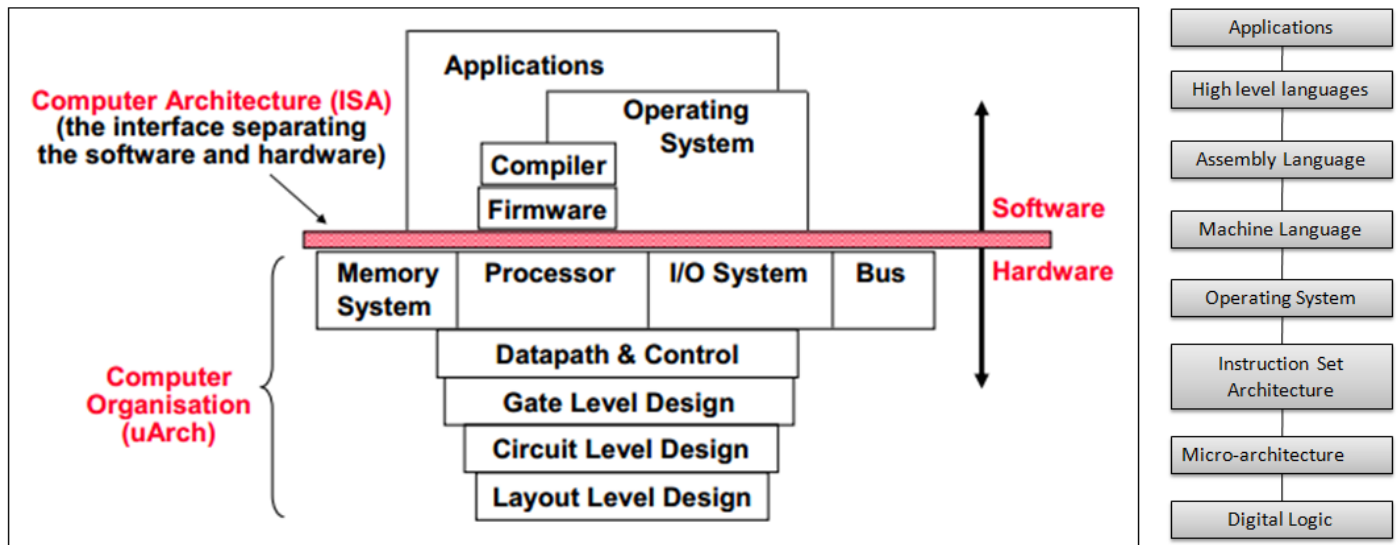


Tutorial 7: Architecture, Procedure Call and Addressing Modes

- Q1. Use a suitable diagram to show the Computer Architecture Levels of abstraction. Then, briefly describe the two main category in this abstraction level.

The typical Computer Architecture Levels of abstraction is shown in figure below.



There are two main categories namely Computer Architecture which consists mainly of software, and Computer Organization which consists of hardware components, or also known as micro-architecture. Their brief description as below.

1. Instruction Set Architecture level : Software perspective of a computer, consists of set of instructions specific for a particular hardware, available for programmer usage to control the operations of a PC. Significant for the *abstraction layer* between the actual hardware and the low-level software
 2. Micro-Architecture : A logical implementation of an ISA by specifying Functional Units includes Structures, Algorithm, Microcode, Register and Error tolerance mechanism.
- Q2. Compare and contrast RISC versus CISC architecture. Hence give TWO reasons why MIPS ISA is chosen for academic study usage.

Comparison of RISC vs CISC is shown in table below. Two reasons why MIPS ISA is chosen as below:

- Simple design and yet high performance as embedded processor
- Large market for embedded apps

	CISC	RISC
1	Variable instruction length	Fixed instruction length
2	Large number of addressing modes	Few addressing modes.
3	Support for small number of general purpose registers.	Support for large number of general purpose registers
4	Requires less number of instructions to represent an application code when compared to RISC.	Requires more number of instructions to represent an application code when compared to CISC although this is debatable
5	Requires complex Compiler.	Requires less complex Compiler
6	Since the application code compiled for CISC instruction set results in less number of instructions we need less memory to store the application binaries in a CISC machine.	Since the application code compiled for RISC instruction set results in more number of instructions (when compared to CISC) we need more memory to store the application binaries in a RISC machine
7	Less number of instructions need not necessarily mean that an application running on a CISC processor results in higher performance than the same running on a RISC processor.	More number of instructions need not necessarily mean that an application running on a RISC processor results in lower performance than the same running on a CISC processor
8	In addition to Load/Store there are other instructions which results in accessing memory.	Load/Store (Atomics included) are the only ones which can access memory
9	A typical CISC instruction (<i>Intel x 86 instruction</i>).	A typical RISC instruction (<i>SPARC instruction</i>)
10	Examples of CISC processors are Intel's 486, Pentium (all flavours), AMD's Krypton, Athlon etc.	Examples of RISC processors are SUN's UltraSparc, MIPS's MIPS32, MIPS64, ARM'S ARM11, Motorola's PowerPC etc

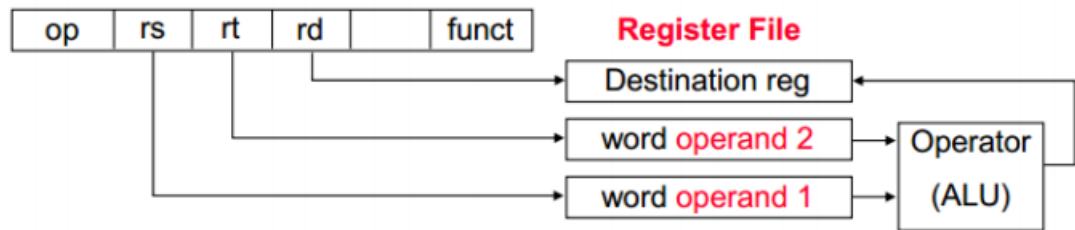
Q3. What is meant by addressing mode? Describe the MIPS Register addressing mode with examples of instruction and a suitable diagram.

Addressing modes

- specify how an architecture determines the address of an operand or instruction in memory
- The actual memory address specified by the addressing mode is called the effective address.

The MIPS R-format Register Addressing modes is elaborated as below:

- The operand is in a register
- Register number in instruction tells which register contains data
- Register may contain data (add) or address (jr)
- used by instructions eg. Arithmetic : add, sub etc.
Logical : and, or, sll, srl, etc
Program Control : slt, sltu, jr, etc



- Q4. Determine the MIPS Addressing Modes as shown in the figure below. Briefly describe this mode and indicate the class of MIPS instruction typically based on this mode.

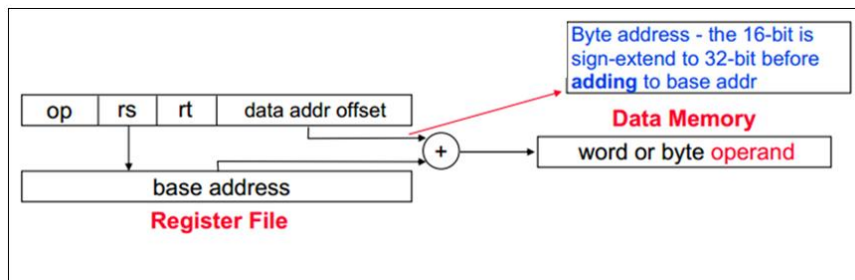


Figure shows (I-Format) Base Addressing Modes.

In Base (or displacement) addressing mode, the operand's memory address is obtained by summing a register (containing an address value acting as the base) and a 16-bit constant contained within the instruction.

This mode is typically used by Data Transfer class of instructions.

- Q5. Below is a procedure *Cal* which compute four integer values. Describe what does compiler do in following section (a) ,(b)... in systematical steps to execute this C code procedure.

```
int main()
{
    printf("The calculated value is %d\n",
        cal(1, 2, 3, 4));
}

int cal(int g, int h, int i, int j)
{
    int f;
    f = (g + h) - (i + j);
    return f;
}
```

a) Steps done by Caller (main function) before passing argument to procedure.

- Caller saves old value on stack
- Caller prepares the arguments (\$a0~\$a3), old arg registers need not save if main doesn't require reuse
- If more arg, push them on the stack
- If caller expect to use one of \$a0~\$a3 after the call, caller must save the value before the call, however, \$t0 ~\$t9 need not be saved

```
addi $sp, $sp, -4
sw    $ra, 0($sp)
addi $a0, $0, 1
addi $a1, $0, 2
addi $a2, $0, 3
addi $a3, $0, 4
```

(b) Steps done by Caller to transfer control to procedure Cal.

- Caller (main function) use instruction jal to transfer control to callee (procedure), it involves *link* and *jump* , => link and jump
- *Link* : h/w saves the next instraddr (PC+4) into \$ra for proc to rtn to caller
- *Jump* : transfer to procaddr specify by the label : cal

```
addi $sp, $sp, -4
sw    $ra, 0($sp)
addi $a0, $0, 1
addi $a1, $0, 2
addi $a2, $0, 3
addi $a3, $0, 4
jal cal
```

(c) Steps done by Callee before execute the Cal function.

- Callee allocate stack frame => cal allocate stack frame
- Callee(calproc) saves \$s0 ~ \$s7, \$ra values onto stack before using to prevent loss of caller's info

```
addi $sp, $sp, -4
sw    $ra, 0($sp)
addi $a0, $0, 1
addi $a1, $0, 2
addi $a2, $0, 3
addi $a3, $0, 4
jal cal
```

```
cal:
addi $sp, $sp, -8
sw    $s1, 4($sp)
sw    $s0, 0($sp)
```

(d) Steps done by Callee to perform it's Cal computation task .

- Callee performs the desired task (arithmetic instr and store into \$v0)
- Callee restores the registers values (which were saved during procedure entry)
- Callee de-allocate stack frame
- Callee returns control to caller (main function)

```
addi $sp, $sp, -4
sw   $ra, 0($sp)
addi $a0, $0, 1
addi $a1, $0, 2
addi $a2, $0, 3
addi $a3, $0, 4
jal  cal
lw   $ra, 0($sp)
addi $sp, $sp, 4
```

```
cal:
addi $sp, $sp, -8
sw   $s1, 4($sp)
sw   $s0, 0($sp)
```

```
add  $s0, $a0, $a1
add  $s1, $a2, $a3
sub  $v0, $s0, $s1
lw   $s0, 0($sp)
lw   $s1, 4($sp)
addi $sp, $sp, 8
jr   $ra
```